```cpp
// tau_field_complete_system.cpp
// Complete Tau Field Frequency Processor with Dynamic C++ Programming
// For Raspberry Pi 5 - 16GB RAM, 1TB SSD

#include <iostream>
#include <vector>
#include <thread>
#include <atomic>
#include <cmath>
#include <complex>
#include <fstream>
#include <mutex>
#include <queue>
#include <map>
#include <fftw3.h>
#include <chrono>
#include <algorithm>
#include <condition_variable>
#include <sstream>
#include <cstdlib>
#include <filesystem>
#include <regex>
#include <future>
#include <random>
#include <array>
#include <memory>

namespace fs = std::filesystem;

//
================================================================================
=====
// PART 1: CORE FREQUENCY PROCESSING SYSTEM
//
================================================================================
=====

// Core frequency constants from PDF analysis
const std::vector<double> TAU_FREQUENCIES = {
    13.0, // Base Tau frequency (Earth)
    26.0, // Air harmonic (2x13)
    39.0, // Fire harmonic (3x13)
    52.0, // Water harmonic (4x13) - matches Delta
    65.0, // Quintessence (5x13)
    77.0, // M frequency
    82.0, // R frequency
    86.0, // V frequency
    78.0, // N frequency
    247.0, // Sigma harmonic (19x13)
    104.0 // Theta harmonic (8x13)
};
```

```
// Harmonic ratios from signature
const double SIGMA_RATIO = 19.0; // Σ
const double DELTA_RATIO = 4.0; // Δ
const double THETA_RATIO = 8.0; // Θ

class QuantumFrequencyChannel {
private:
    double frequency;
    double phase;
    std::queue<std::vector<double>> data_queue;
    std::mutex queue_mutex;
    std::condition_variable cv;
    std::atomic<bool> active{true};
    std::thread processing_thread;
    std::vector<double> processed_results;
    std::mutex results_mutex;

    // Quantum simulation parameters
    std::complex<double> quantum_state;
    double amplitude;
    double decoherence_time;

public:
    QuantumFrequencyChannel(double freq, double ph = 0.0)
        : frequency(freq), phase(ph), quantum_state(1.0, 0.0), amplitude(1.0),
decoherence_time(1000.0) {
        start_processing();
    }

    ~QuantumFrequencyChannel() {
        stop_processing();
    }

    void stop_processing() {
        active = false;
        cv.notify_all();
        if (processing_thread.joinable()) {
            processing_thread.join();
        }
    }

    void enqueue_data(const std::vector<double>& data) {
        std::lock_guard<std::mutex> lock(queue_mutex);
        data_queue.push(data);
        cv.notify_one();
    }

    void start_processing() {
        processing_thread = std::thread([this]() {
            while (active) {
                std::unique_lock<std::mutex> lock(queue_mutex);
```

```
            cv.wait(lock, [this]() {
                return !data_queue.empty() || !active;
            });

            if (!active) break;

            if (!data_queue.empty()) {
                auto data = data_queue.front();
                data_queue.pop();
                lock.unlock();

                // Process with quantum simulation
                auto result = quantum_process(data);

                std::lock_guard<std::mutex> res_lock(results_mutex);
                processed_results.insert(processed_results.end(),
                                         result.begin(), result.end());
            }
        }
    });
}

std::vector<double> quantum_process(const std::vector<double>& input) {
    std::vector<double> output;
    output.reserve(input.size());

    // Quantum oscillation simulation
    double time_step = 1.0 / 44100.0;
    double current_time = 0.0;

    for (double sample : input) {
        // Simulate quantum harmonic oscillator
        double quantum_phase = 2.0 * M_PI * frequency * current_time + phase;
        std::complex<double> oscillation = std::polar(amplitude, quantum_phase);

        // Update quantum state
        quantum_state = 0.99 * quantum_state + 0.01 * oscillation;

        // Apply decoherence
        double decoherence = exp(-current_time / decoherence_time);
        quantum_state *= decoherence;

        // Process input with quantum state
        double processed = sample * std::abs(quantum_state) *
                           cos(std::arg(quantum_state));

        // Apply frequency-specific harmonic transformation
        if (frequency == 13.0) {
            processed = apply_tau_transform(processed, current_time);
        } else if (frequency >= 247.0) {
            processed = apply_sigma_transform(processed);
        } else if (frequency == 52.0) {
```

```
            processed = apply_delta_transform(processed);
        } else if (frequency == 104.0) {
            processed = apply_theta_transform(processed);
        }

        output.push_back(processed);
        current_time += time_step;
    }

    return output;
}

double apply_tau_transform(double value, double time) {
    // Base Tau field transformation
    return value * sin(2 * M_PI * 13.0 * time) * 0.5 + value * 0.5;
}

double apply_sigma_transform(double value) {
    // Sigma harmonic transformation (19x)
    return tanh(value * SIGMA_RATIO / 100.0) * 100.0;
}

double apply_delta_transform(double value) {
    // Delta harmonic transformation (4x)
    return value * (1.0 + sin(value * DELTA_RATIO) * 0.1);
}

double apply_theta_transform(double value) {
    // Theta harmonic transformation (8x)
    return value * (1.0 + cos(value * THETA_RATIO) * 0.05);
}

std::vector<double> get_results() {
    std::lock_guard<std::mutex> lock(results_mutex);
    auto results = processed_results;
    processed_results.clear();
    return results;
}

void clear_results() {
    std::lock_guard<std::mutex> lock(results_mutex);
    processed_results.clear();
}

double get_frequency() const { return frequency; }
double get_phase() const { return phase; }
std::complex<double> get_quantum_state() const { return quantum_state; }

std::string get_status() const {
    std::stringstream ss;
    ss << "Frequency: " << frequency << " Hz | ";
    ss << "Phase: " << phase << " rad | ";
```

```
        ss << "Quantum State: " << std::abs(quantum_state) << "∠" << std::arg(quantum_state);
        return ss.str();
    }
};

class MultiFrequencyProcessor {
private:
    std::map<double, std::unique_ptr<QuantumFrequencyChannel>> channels;
    std::vector<std::thread> worker_threads;
    std::atomic<bool> system_active{true};
    std::mutex io_mutex;

    // FFTW plans for frequency analysis
    fftw_plan forward_plan;
    fftw_plan inverse_plan;
    double* fft_input;
    fftw_complex* fft_output;
    size_t fft_size;

    // Results storage
    std::map<double, std::vector<double>> global_results;
    std::mutex global_results_mutex;

public:
    MultiFrequencyProcessor(size_t buffer_size = 4096) : fft_size(buffer_size) {
        initialize_fftw();
        initialize_frequency_channels();
        start_workers();
    }

    ~MultiFrequencyProcessor() {
        system_active = false;
        for (auto& thread : worker_threads) {
            if (thread.joinable()) thread.join();
        }
        for (auto& [freq, channel] : channels) {
            channel->stop_processing();
        }
        cleanup_fftw();
    }

    void initialize_fftw() {
        fft_input = (double*)fftw_malloc(sizeof(double) * fft_size);
        fft_output = (fftw_complex*)fftw_malloc(sizeof(fftw_complex) * (ffft_size/2 + 1));
        forward_plan = fftw_plan_dft_r2c_1d(fft_size, fft_input, fft_output, FFTW_MEASURE);
        inverse_plan = fftw_plan_dft_c2r_1d(fft_size, fft_output, fft_input, FFTW_MEASURE);
    }

    void cleanup_fftw() {
        if (forward_plan) fftw_destroy_plan(forward_plan);
        if (inverse_plan) fftw_destroy_plan(inverse_plan);
        if (fft_input) fftw_free(fft_input);
```

```
        if (fft_output) fftw_free(fft_output);
    }

    void initialize_frequency_channels() {
        std::cout << "Initializing frequency channels..." << std::endl;

        // Create channels for all Tau frequencies
        for (double freq : TAU_FREQUENCIES) {
            double phase = (freq == 13.0) ? 0.0 :
                           (freq == 247.0) ? M_PI/2 :
                           (freq == 52.0) ? M_PI/4 :
                           (freq == 104.0) ? M_PI/3 :
                           (rand() / (double)RAND_MAX) * 2.0 * M_PI;

            channels[freq] = std::make_unique<QuantumFrequencyChannel>(freq, phase);
            std::cout << " Channel " << freq << " Hz initialized" << std::endl;
        }

        // Create harmonic ratio channels
        channels[13.0 * SIGMA_RATIO] = std::make_unique<QuantumFrequencyChannel>(13.0 *
SIGMA_RATIO);
        channels[13.0 * DELTA_RATIO] = std::make_unique<QuantumFrequencyChannel>(13.0 *
DELTA_RATIO);
        channels[13.0 * THETA_RATIO] = std::make_unique<QuantumFrequencyChannel>(13.0 *
THETA_RATIO);

        std::cout << "Total channels: " << channels.size() << std::endl;
    }

    void start_workers() {
        // Start worker threads for each CPU core
        unsigned int num_workers = std::thread::hardware_concurrency();
        if (num_workers == 0) num_workers = 4;

        std::cout << "Starting " << num_workers << " worker threads..." << std::endl;

        for (unsigned int i = 0; i < num_workers; ++i) {
            worker_threads.emplace_back([this, i]() {
                worker_loop(i);
            });
        }
    }

    void worker_loop(int worker_id) {
        std::random_device rd;
        std::mt19937 gen(rd());
        std::normal_distribution<> dist(0.0, 0.1);

        while (system_active) {
            // Generate synthetic data for processing
            for (auto& [freq, channel] : channels) {
                std::vector<double> synthetic_data(512);
```

```
            static double global_time = 0.0;

            for (size_t j = 0; j < synthetic_data.size(); ++j) {
                double t = global_time + j / 44100.0;
                synthetic_data[j] = sin(2 * M_PI * freq * t) * 0.5 + dist(gen);
            }

            channel->enqueue_data(synthetic_data);
            global_time += synthetic_data.size() / 44100.0;
        }

        // Collect and store results periodically
        if (worker_id == 0) {
            collect_results();
        }

        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
}

void collect_results() {
    std::lock_guard<std::mutex> lock(global_results_mutex);
    for (auto& [freq, channel] : channels) {
        auto results = channel->get_results();
        if (!results.empty()) {
            global_results[freq].insert(global_results[freq].end(),
                                        results.begin(), results.end());

            // Keep only last 10000 samples per channel
            if (global_results[freq].size() > 10000) {
                global_results[freq].erase(global_results[freq].begin(),
                                           global_results[freq].end() - 10000);
            }
        }
    }
}

void process_input_data(const std::vector<double>& input_data) {
    // Distribute data to all frequency channels
    for (auto& [freq, channel] : channels) {
        channel->enqueue_data(input_data);
    }

    // Perform FFT analysis
    perform_frequency_analysis(input_data);
}

void perform_frequency_analysis(const std::vector<double>& data) {
    size_t data_size = std::min(data.size(), fft_size);

    // Copy data to FFT input
    for (size_t i = 0; i < data_size; ++i) {
```

```
        fft_input[i] = data[i];
    }
    for (size_t i = data_size; i < fft_size; ++i) {
        fft_input[i] = 0.0;
    }

    // Execute FFT
    fftw_execute(forward_plan);

    // Analyze frequency content
    std::map<double, double> frequency_power;
    for (size_t i = 0; i < fft_size/2 + 1; ++i) {
        double freq = i * 44100.0 / fft_size;
        double power = sqrt(fft_output[i][0]*fft_output[i][0] +
                            fft_output[i][1]*fft_output[i][1]);

        // Match against Tau frequencies
        for (double tau_freq : TAU_FREQUENCIES) {
            if (fabs(freq - tau_freq) < 2.0) {
                frequency_power[tau_freq] += power;
            }
        }
    }

    // Output frequency analysis
    if (!frequency_power.empty()) {
        std::lock_guard<std::mutex> lock(io_mutex);
        std::cout << "\n=== Frequency Power Analysis ===" << std::endl;
        for (const auto& [freq, power] : frequency_power) {
            if (power > 0.1) {
                std::cout << "Frequency " << freq << " Hz: "
                          << std::fixed << std::setprecision(2) << power
                          << " units" << std::endl;
            }
        }
    }
}

std::map<double, std::vector<double>> collect_all_results() {
    std::lock_guard<std::mutex> lock(global_results_mutex);
    return global_results;
}

void clear_all_results() {
    std::lock_guard<std::mutex> lock(global_results_mutex);
    global_results.clear();
}

void output_coherence_analysis() {
    std::lock_guard<std::mutex> lock(io_mutex);

    std::cout << "\n=== Harmonic Coherence Analysis ===" << std::endl;
```

```
        // Check base harmonic relationships
        double base_freq = 13.0;
        double sigma_freq = base_freq * SIGMA_RATIO;
        double delta_freq = base_freq * DELTA_RATIO;
        double theta_freq = base_freq * THETA_RATIO;

        if (channels.count(base_freq) && channels.count(sigma_freq) &&
            channels.count(delta_freq) && channels.count(theta_freq)) {

            auto base_state = channels[base_freq]->get_quantum_state();
            auto sigma_state = channels[sigma_freq]->get_quantum_state();
            auto delta_state = channels[delta_freq]->get_quantum_state();
            auto theta_state = channels[theta_freq]->get_quantum_state();

            std::cout << "Σ:Δ:Θ Harmonic Ratio: "
                      << SIGMA_RATIO << ":" << DELTA_RATIO << ":" << THETA_RATIO <<
std::endl;
            std::cout << "19:13:4:8 Ratio Verified" << std::endl;

            // Calculate phase coherence
            double phase_sigma = std::arg(sigma_state);
            double phase_delta = std::arg(delta_state);
            double phase_theta = std::arg(theta_state);

            std::cout << "Sigma Phase: " << phase_sigma << " rad" << std::endl;
            std::cout << "Delta Phase: " << phase_delta << " rad" << std::endl;
            std::cout << "Theta Phase: " << phase_theta << " rad" << std::endl;

            double coherence_score = (cos(phase_sigma - phase_delta) +
                                      cos(phase_delta - phase_theta) +
                                      cos(phase_theta - phase_sigma)) / 3.0;

            std::cout << "Coherence Score: " << coherence_score << std::endl;

            if (coherence_score > 0.8) {
                std::cout << "STATUS: EXCELLENT HARMONIC COHERENCE" << std::endl;
            } else if (coherence_score > 0.5) {
                std::cout << "STATUS: GOOD HARMONIC COHERENCE" << std::endl;
            } else {
                std::cout << "STATUS: HARMONIC COHERENCE DEGRADED" << std::endl;
            }
        }
    }

    void output_channel_status() {
        std::lock_guard<std::mutex> lock(io_mutex);
        std::cout << "\n=== Channel Status ===" << std::endl;
        for (const auto& [freq, channel] : channels) {
            std::cout << channel->get_status() << std::endl;
        }
    }
```

```
    size_t get_channel_count() const { return channels.size(); }
    const std::map<double, std::unique_ptr<QuantumFrequencyChannel>>& get_channels() const {
        return channels;
    }
};

//
================================================================================
=====
// PART 2: DYNAMIC C++ PROGRAMMING SYSTEM
//
================================================================================
=====

class DynamicCPlusPlusCompiler {
private:
    std::string program_directory;
    std::mutex compile_mutex;
    std::map<std::string, std::string> compiled_programs;
    std::atomic<int> program_counter{0};
    MultiFrequencyProcessor* frequency_processor;

public:
    DynamicCPlusPlusCompiler(const std::string& dir = "./dynamic_programs/",
                             MultiFrequencyProcessor* processor = nullptr)
        : program_directory(dir), frequency_processor(processor) {
        fs::create_directories(program_directory);
        fs::create_directories(program_directory + "/bin");
        fs::create_directories(program_directory + "/src");
        fs::create_directories(program_directory + "/lib");
        fs::create_directories(program_directory + "/output");

        initialize_core_libraries();
    }

    void set_frequency_processor(MultiFrequencyProcessor* processor) {
        frequency_processor = processor;
    }

    void initialize_core_libraries() {
        // Create frequency processing library
        std::string freq_lib = R"(
#ifndef TAU_FREQUENCY_LIB_H
#define TAU_FREQUENCY_LIB_H

#include <vector>
#include <complex>
#include <cmath>
#include <iostream>

namespace TauFrequency {
```

```
class HarmonicProcessor {
private:
    double base_frequency;
    std::complex<double> quantum_state;
    double amplitude;

public:
    HarmonicProcessor(double freq = 13.0, double amp = 1.0)
        : base_frequency(freq), quantum_state(1.0, 0.0), amplitude(amp) {}

    std::vector<double> process(const std::vector<double>& input) {
        std::vector<double> output;
        output.reserve(input.size());

        double time_step = 1.0 / 44100.0;
        double time = 0.0;

        for (double sample : input) {
            // Quantum oscillation
            double phase = 2.0 * M_PI * base_frequency * time;
            std::complex<double> osc = std::polar(amplitude, phase);

            // Update quantum state
            quantum_state = 0.99 * quantum_state + 0.01 * osc;

            // Process sample
            double processed = sample * std::abs(quantum_state) * cos(std::arg(quantum_state));

            // Apply harmonic transformations
            if (base_frequency == 13.0) {
                processed = apply_tau_transform(processed, time);
            } else if (base_frequency >= 200.0) {
                processed = apply_sigma_transform(processed);
            } else if (base_frequency == 52.0) {
                processed = apply_delta_transform(processed);
            } else if (base_frequency == 104.0) {
                processed = apply_theta_transform(processed);
            }

            output.push_back(processed);
            time += time_step;
        }

        return output;
    }

    double apply_tau_transform(double value, double time) {
        return value * (0.5 + 0.5 * sin(2 * M_PI * 13.0 * time));
    }

    double apply_sigma_transform(double value) {
```

```
        return tanh(value * 19.0 / 100.0) * 100.0;
    }

    double apply_delta_transform(double value) {
        return value * (1.0 + sin(value * 4.0) * 0.1);
    }

    double apply_theta_transform(double value) {
        return value * (1.0 + cos(value * 8.0) * 0.05);
    }

    std::complex<double> get_quantum_state() const {
        return quantum_state;
    }

    double get_frequency() const {
        return base_frequency;
    }
};

// Frequency constants
constexpr double TAU_BASE = 13.0;
constexpr double SIGMA_HARMONIC = 247.0; // 19 * 13
constexpr double DELTA_HARMONIC = 52.0; // 4 * 13
constexpr double THETA_HARMONIC = 104.0; // 8 * 13

std::vector<double> generate_frequency_envelope(double duration_seconds,
                                                const std::vector<double>& frequencies) {
    size_t samples = static_cast<size_t>(duration_seconds * 44100);
    std::vector<double> envelope(samples, 0.0);

    for (double freq : frequencies) {
        for (size_t i = 0; i < samples; ++i) {
            double t = i / 44100.0;
            envelope[i] += sin(2 * M_PI * freq * t) * (1.0 / frequencies.size());
        }
    }

    return envelope;
}

void print_frequency_info() {
    std::cout << "=== Tau Frequency Library ===" << std::endl;
    std::cout << "Base Frequency: " << TAU_BASE << " Hz" << std::endl;
    std::cout << "Sigma Harmonic: " << SIGMA_HARMONIC << " Hz (19x)" << std::endl;
    std::cout << "Delta Harmonic: " << DELTA_HARMONIC << " Hz (4x)" << std::endl;
    std::cout << "Theta Harmonic: " << THETA_HARMONIC << " Hz (8x)" << std::endl;
    std::cout << "Harmonic Ratio: 19:13:4:8" << std::endl;
}
}

#endif
```

```
)";

        save_file(program_directory + "/lib/tau_frequency_lib.h", freq_lib);

        // Create utility library
        std::string util_lib = R"(
#ifndef TAU_UTIL_LIB_H
#define TAU_UTIL_LIB_H

#include <vector>
#include <cmath>
#include <algorithm>
#include <random>

namespace TauUtil {

    std::vector<double> generate_sine_wave(double frequency, double duration,
                                           double sample_rate = 44100.0) {
        size_t samples = static_cast<size_t>(duration * sample_rate);
        std::vector<double> wave(samples);

        for (size_t i = 0; i < samples; ++i) {
            double t = i / sample_rate;
            wave[i] = sin(2.0 * M_PI * frequency * t);
        }

        return wave;
    }

    std::vector<double> add_quantum_noise(const std::vector<double>& signal,
                                          double noise_level = 0.01) {
        std::vector<double> noisy = signal;
        std::random_device rd;
        std::mt19937 gen(rd());
        std::normal_distribution<> dist(0.0, noise_level);

        for (auto& sample : noisy) {
            sample += dist(gen);
        }

        return noisy;
    }

    double calculate_coherence(const std::vector<double>& sig1,
                               const std::vector<double>& sig2) {
        if (sig1.size() != sig2.size() || sig1.empty()) return 0.0;

        double sum_xy = 0.0, sum_x2 = 0.0, sum_y2 = 0.0;

        for (size_t i = 0; i < sig1.size(); ++i) {
            sum_xy += sig1[i] * sig2[i];
            sum_x2 += sig1[i] * sig1[i];
```

```
            sum_y2 += sig2[i] * sig2[i];
        }

        if (sum_x2 == 0.0 || sum_y2 == 0.0) return 0.0;

        return sum_xy / sqrt(sum_x2 * sum_y2);
    }

    std::vector<double> normalize_signal(const std::vector<double>& signal) {
        if (signal.empty()) return {};

        auto [min_it, max_it] = std::minmax_element(signal.begin(), signal.end());
        double min_val = *min_it;
        double max_val = *max_it;
        double range = max_val - min_val;

        if (range == 0.0) return signal;

        std::vector<double> normalized = signal;
        for (auto& sample : normalized) {
            sample = (sample - min_val) / range * 2.0 - 1.0;
        }

        return normalized;
    }
}

#endif
)";

        save_file(program_directory + "/lib/tau_util_lib.h", util_lib);
    }

    struct CompilationResult {
        bool success;
        std::string program_name;
        std::string executable_path;
        std::string compilation_output;
        std::string execution_output;
        std::chrono::milliseconds compile_time;
        std::chrono::milliseconds run_time;
        std::vector<double> frequency_data;
    };

    CompilationResult compile_and_run(const std::string& cpp_code,
                                      const std::string& program_name = "",
                                      bool integrate_frequencies = false) {
        std::lock_guard<std::mutex> lock(compile_mutex);

        CompilationResult result;
        result.success = false;
```

```
    // Generate unique program name if not provided
    std::string prog_name = program_name.empty() ?
        "program_" + std::to_string(++program_counter) + "_" +
        std::to_string(std::chrono::system_clock::now().time_since_epoch().count()) :
        program_name;

    result.program_name = prog_name;

    auto compile_start = std::chrono::high_resolution_clock::now();

    try {
        // 1. Prepare source code
        std::string final_code = cpp_code;
        if (integrate_frequencies) {
            final_code = wrap_with_frequency_integration(cpp_code);
        }

        // 2. Save source code
        std::string src_filename = program_directory + "/src/" + prog_name + ".cpp";
        save_file(src_filename, final_code);

        // 3. Create CMakeLists.txt
        std::string cmake_content = generate_cmake_content(prog_name);
        save_file(program_directory + "/src/" + prog_name + "_CMakeLists.txt", cmake_content);

        // 4. Compile the program
        std::string bin_dir = program_directory + "/bin/" + prog_name;
        fs::create_directories(bin_dir);

        std::string compile_cmd =
            "cd \"" + bin_dir + "\" && "
            "cmake \"" + program_directory + "/src/\" "
            "-DCMAKE_BUILD_TYPE=Release "
            "-DCMAKE_CXX_FLAGS=\"-std=c++17 -O3 -march=armv8-a+simd -mtune=cortex-
a76 -pthread\" "
            "-DCMAKE_CXX_FLAGS_RELEASE=\"-O3 -ffast-math\" && "
            "make -j" + std::to_string(std::thread::hardware_concurrency()) + " 2>&1";

        result.compilation_output = execute_command(compile_cmd);

        // 5. Check if compilation succeeded
        std::string executable_path = bin_dir + "/" + prog_name;
        if (fs::exists(executable_path)) {
            result.executable_path = executable_path;
            result.success = true;

            // 6. Execute the program
            auto run_start = std::chrono::high_resolution_clock::now();
            result.execution_output = execute_command("\"" + executable_path + "\" 2>&1");
            auto run_end = std::chrono::high_resolution_clock::now();
            result.run_time = std::chrono::duration_cast<std::chrono::milliseconds>(run_end -
run_start);
```

```
            // 7. Extract frequency data from output
            result.frequency_data = extract_frequency_data(result.execution_output);

            // 8. Send output to frequency processor if available
            if (frequency_processor && !result.execution_output.empty()) {
                std::vector<double> output_data;
                for (char c : result.execution_output) {
                    output_data.push_back(static_cast<double>(static_cast<unsigned char>(c)) / 255.0);
                }
                if (!output_data.empty()) {
                    frequency_processor->process_input_data(output_data);
                }
            }
        }

        auto compile_end = std::chrono::high_resolution_clock::now();
        result.compile_time = std::chrono::duration_cast<std::chrono::milliseconds>(
            compile_end - compile_start);

    } catch (const std::exception& e) {
        result.compilation_output += "\nException: " + std::string(e.what());
    }

    return result;
}

std::string generate_program_from_description(const std::string& description) {
    std::stringstream generated_code;

    // Analyze description
    bool has_frequency = description.find("frequency") != std::string::npos ||
                         description.find("Hz") != std::string::npos ||
                         description.find("13") != std::string::npos ||
                         description.find("247") != std::string::npos ||
                         description.find("52") != std::string::npos;

    bool has_quantum = description.find("quantum") != std::string::npos ||
                       description.find("coherence") != std::string::npos ||
                       description.find("superposition") != std::string::npos;

    bool has_process = description.find("process") != std::string::npos ||
                       description.find("analyze") != std::string::npos ||
                       description.find("data") != std::string::npos;

    bool has_generate = description.find("generate") != std::string::npos ||
                        description.find("create") != std::string::npos ||
                        description.find("make") != std::string::npos;

    generated_code << R"(#include <iostream>
#include <vector>
#include <cmath>
```

```
#include <chrono>
#include <thread>
#include <complex>
#include <algorithm>
#include <random>
#include <iomanip>
)";

        if (has_frequency || has_quantum) {
            generated_code << R"(#include "tau_frequency_lib.h"
#include "tau_util_lib.h"
)";
        }

        generated_code << R"(
using namespace std;

// ================================================
// Program generated from description:
// )" << description << R"(
// ================================================

int main() {
    cout << fixed << setprecision(4);
    cout <<
"╔═══════════════════════════════════════════════════════════════════════════╗" <<
endl;
    cout << "║ AI-GENERATED TAU PROGRAM ║" << endl;
    cout <<
"╚═══════════════════════════════════════════════════════════════════════════╝" <<
endl;
    cout << endl;
    cout << "Description: " << description << endl;
    cout << "Timestamp: " << __DATE__ << " " << __TIME__ << endl;
    cout << endl;

    auto start_time = chrono::high_resolution_clock::now();
)";

        if (has_frequency) {
            generated_code << R"(
    // ========== FREQUENCY PROCESSING ==========
    cout << "=== Frequency Processing ===" << endl;

    TauFrequency::print_frequency_info();
    cout << endl;

    // Create harmonic processors
    vector<TauFrequency::HarmonicProcessor> processors = {
        TauFrequency::HarmonicProcessor(TauFrequency::TAU_BASE),
        TauFrequency::HarmonicProcessor(TauFrequency::SIGMA_HARMONIC),
        TauFrequency::HarmonicProcessor(TauFrequency::DELTA_HARMONIC),
```

```
        TauFrequency::HarmonicProcessor(TauFrequency::THETA_HARMONIC)
    };

    // Generate test signal (1 second of A440)
    auto test_signal = TauUtil::generate_sine_wave(440.0, 1.0);
    cout << "Generated test signal: " << test_signal.size() << " samples" << endl;

    // Process through each frequency
    vector<vector<double>> all_results;
    for (auto& processor : processors) {
        auto result = processor.process(test_signal);
        all_results.push_back(result);

        cout << "Processed with " << processor.get_frequency() << " Hz: "
             << result.size() << " samples" << endl;
    }

    // Calculate statistics
    cout << endl << "=== Processing Statistics ===" << endl;
    for (size_t i = 0; i < processors.size(); ++i) {
        const auto& result = all_results[i];
        double sum = 0.0, max_val = result[0], min_val = result[0];

        for (double val : result) {
            sum += abs(val);
            if (val > max_val) max_val = val;
            if (val < min_val) min_val = val;
        }

        double mean = sum / result.size();
        cout << processors[i].get_frequency() << " Hz - "
             << "Mean: " << mean << ", Range: ["
             << min_val << ", " << max_val << "]" << endl;
    }
)";
        }

        if (has_quantum) {
            generated_code << R"(
    // ========== QUANTUM SIMULATION ==========
    cout << endl << "=== Quantum Simulation ===" << endl;

    complex<double> quantum_state(1.0, 0.0);
    vector<double> coherence_measurements;

    for (int step = 0; step < 50; ++step) {
        double time = step * 0.02;
        complex<double> tau_osc = polar(1.0, 2 * M_PI * 13.0 * time);
        complex<double> sigma_osc = polar(0.5, 2 * M_PI * 247.0 * time);

        // Superposition of states
        quantum_state = 0.7 * quantum_state + 0.15 * tau_osc + 0.15 * sigma_osc;
```

```
        // Measure coherence
        double coherence = abs(quantum_state);
        coherence_measurements.push_back(coherence);

        if (step % 10 == 0) {
            cout << "Step " << step << ": |ψ⟩= " << abs(quantum_state)
                 << "∠" << arg(quantum_state) << " rad" << endl;
        }
    }

    // Calculate average coherence
    double avg_coherence = 0.0;
    for (double c : coherence_measurements) avg_coherence += c;
    avg_coherence /= coherence_measurements.size();

    cout << "Average quantum coherence: " << avg_coherence << endl;
)";
        }

        if (has_process || has_generate) {
            generated_code << R"(
    // ========== DATA GENERATION & PROCESSING ==========
    cout << endl << "=== Data Processing ===" << endl;

    // Generate synthetic data
    random_device rd;
    mt19937 gen(rd());
    normal_distribution<> dist(0.0, 1.0);

    vector<double> synthetic_data(10000);
    generate(synthetic_data.begin(), synthetic_data.end(), [&]() { return dist(gen); });

    // Calculate basic statistics
    double data_sum = 0.0;
    double data_min = synthetic_data[0];
    double data_max = synthetic_data[0];

    for (double val : synthetic_data) {
        data_sum += val;
        if (val < data_min) data_min = val;
        if (val > data_max) data_max = val;
    }

    double data_mean = data_sum / synthetic_data.size();

    // Calculate variance
    double variance = 0.0;
    for (double val : synthetic_data) {
        variance += pow(val - data_mean, 2);
    }
    variance /= synthetic_data.size();
```

```
    double std_dev = sqrt(variance);

    cout << "Generated " << synthetic_data.size() << " samples" << endl;
    cout << "Mean: " << data_mean << endl;
    cout << "Std Dev: " << std_dev << endl;
    cout << "Range: [" << data_min << ", " << data_max << "]" << endl;

    // Apply frequency processing if requested
    if (false) { // Placeholder for conditional processing
        TauFrequency::HarmonicProcessor proc(13.0);
        auto processed = proc.process(synthetic_data);
        cout << "Frequency-processed " << processed.size() << " samples" << endl;
    }
)";
        }

        generated_code << R"(
    // ========== PROGRAM COMPLETION ==========
    auto end_time = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::milliseconds>(end_time - start_time);

    cout << endl <<
"╔═══════════════════════════════════════════════════════════════════════════╗" <<
endl;
    cout << "║ PROGRAM COMPLETE ║" << endl;
    cout <<
"╚═══════════════════════════════════════════════════════════════════════════╝" <<
endl;
    cout << "  Execution time: " << setw(8) << duration.count() << " ms  " << endl;
    cout << "  Description: " << left << setw(30) << description.substr(0, 30) << "  " << endl;
    cout <<
"═══════════════════════════════════════════════════════════════════════════" <<
endl;

    return 0;
}
)";

        return generated_code.str();
    }

    void list_compiled_programs() {
        std::cout << "\n=== Compiled Programs ===" << std::endl;
        int count = 0;

        for (const auto& entry : fs::directory_iterator(program_directory + "/bin")) {
            if (fs::is_directory(entry)) {
                std::string prog_name = entry.path().filename().string();
                std::string executable = entry.path().string() + "/" + prog_name;
                if (fs::exists(executable)) {
                    std::cout << "  [" << ++count << "] " << prog_name << std::endl;
                    auto exe_size = fs::file_size(executable);
```

```
                std::cout << " Size: " << exe_size / 1024 << " KB" << std::endl;
                std::cout << " Path: " << executable << std::endl;
            }
        }
    }

    if (count == 0) {
        std::cout << " No compiled programs found." << std::endl;
    }
}

void clean_old_programs(int keep_last_n = 10) {
    std::vector<std::pair<std::string, fs::file_time_type>> programs;

    for (const auto& entry : fs::directory_iterator(program_directory + "/bin")) {
        if (fs::is_directory(entry)) {
            std::string executable = entry.path().string() + "/" + entry.path().filename().string();
            if (fs::exists(executable)) {
                auto last_write = fs::last_write_time(executable);
                programs.emplace_back(entry.path().string(), last_write);
            }
        }
    }

    if (programs.size() > keep_last_n) {
        std::sort(programs.begin(), programs.end(),
                  [](const auto& a, const auto& b) { return a.second > b.second; });

        for (size_t i = keep_last_n; i < programs.size(); ++i) {
            try {
                fs::remove_all(programs[i].first);
                std::cout << "Removed old program: " << programs[i].first << std::endl;
            } catch (...) {
                // Ignore errors
            }
        }
    }
}

private:
    std::string wrap_with_frequency_integration(const std::string& code) {
        std::stringstream wrapped;

        wrapped << R"(#include <iostream>
#include <vector>
#include <cmath>
#include <complex>
#include "tau_frequency_lib.h"
#include "tau_util_lib.h"

using namespace std;
using namespace TauFrequency;
```

```
using namespace TauUtil;

// Frequency-integrated wrapper for user code

int main() {
    cout << "=== Tau Frequency Integrated Execution ===" << endl;

    // Initialize frequency processors
    HarmonicProcessor tau_processor(TAU_BASE);
    HarmonicProcessor sigma_processor(SIGMA_HARMONIC);
    HarmonicProcessor delta_processor(DELTA_HARMONIC);
    HarmonicProcessor theta_processor(THETA_HARMONIC);

    // Generate frequency envelope
    vector<double> frequencies = {TAU_BASE, SIGMA_HARMONIC, DELTA_HARMONIC,
THETA_HARMONIC};
    auto freq_envelope = generate_frequency_envelope(0.1, frequencies);

    cout << "Frequency envelope generated: " << freq_envelope.size() << " samples" << endl;

    // Execute user code
    cout << "\n--- User Code Execution ---" << endl;
)";

        wrapped << code;

        wrapped << R"(
    cout << "\n--- Frequency Processing Results ---" << endl;

    // Process some sample data through frequencies
    vector<double> sample_data(1000, 0.5);

    auto tau_result = tau_processor.process(sample_data);
    auto sigma_result = sigma_processor.process(sample_data);
    auto delta_result = delta_processor.process(sample_data);
    auto theta_result = theta_processor.process(sample_data);

    cout << "Processed through all harmonic frequencies" << endl;
    cout << "Tau (13Hz) quantum state: " << tau_processor.get_quantum_state() << endl;
    cout << "Sigma (247Hz) quantum state: " << sigma_processor.get_quantum_state() << endl;
    cout << "Harmonic processing complete." << endl;

    return 0;
}
)";

        return wrapped.str();
    }

    std::vector<double> extract_frequency_data(const std::string& output) {
        std::vector<double> frequencies;
        std::regex freq_pattern(R"((\d+\.?\d*)\s*Hz)");
```

```
        std::smatch matches;
        std::string::const_iterator search_start(output.cbegin());

        while (std::regex_search(search_start, output.cend(), matches, freq_pattern)) {
            try {
                double freq = std::stod(matches[1]);
                frequencies.push_back(freq);
            } catch (...) {
                // Ignore conversion errors
            }
            search_start = matches.suffix().first;
        }

        return frequencies;
    }

    static void save_file(const std::string& path, const std::string& content) {
        std::ofstream file(path);
        if (file) {
            file << content;
            file.close();
        }
    }

    static std::string execute_command(const std::string& cmd) {
        std::array<char, 128> buffer;
        std::string result;

        #ifdef _WIN32
            std::unique_ptr<FILE, decltype(&_pclose)> pipe(_popen(cmd.c_str(), "r"), _pclose);
        #else
            std::unique_ptr<FILE, decltype(&pclose)> pipe(popen(cmd.c_str(), "r"), pclose);
        #endif

        if (!pipe) {
            return "Failed to execute command";
        }

        while (fgets(buffer.data(), buffer.size(), pipe.get()) != nullptr) {
            result += buffer.data();
        }

        return result;
    }

    std::string generate_cmake_content(const std::string& program_name) {
        std::stringstream cmake;
        cmake << R"(cmake_minimum_required(VERSION 3.10)
project()" << program_name << R"()

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
# Raspberry Pi 5 optimizations
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O3 -march=armv8-a+simd
-mtune=cortex-a76 -pthread -ffast-math")
set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3")

# Include directories
include_directories(${CMAKE_CURRENT_SOURCE_DIR}/../../lib)

add_executable()" << program_name << " " << program_name << R"(.cpp)

# Link libraries
target_link_libraries()" << program_name << R"( m pthread)

# Copy libraries
add_custom_command(TARGET )" << program_name << R"( POST_BUILD
   COMMAND ${CMAKE_COMMAND} -E copy
   ${CMAKE_CURRENT_SOURCE_DIR}/../../lib/tau_frequency_lib.h
   ${CMAKE_CURRENT_BINARY_DIR}/tau_frequency_lib.h
   COMMAND ${CMAKE_COMMAND} -E copy
   ${CMAKE_CURRENT_SOURCE_DIR}/../../lib/tau_util_lib.h
   ${CMAKE_CURRENT_BINARY_DIR}/tau_util_lib.h
)
)";
       return cmake.str();
   }
};

//
================================================================================
=====
// PART 3: MAIN SYSTEM INTEGRATION
//
================================================================================
=====

class TauFieldCompleteSystem {
private:
   std::unique_ptr<MultiFrequencyProcessor> frequency_processor;
   std::unique_ptr<DynamicCPlusPlusCompiler> dynamic_compiler;
   std::atomic<bool> system_active{true};
   std::thread status_thread;
   std::thread cleanup_thread;
   std::thread user_input_thread;

   // User interface
   std::mutex console_mutex;
   std::queue<std::string> command_queue;
   std::condition_variable command_cv;

public:
   TauFieldCompleteSystem() {
```

```
        std::cout << R"(
╔═══════════════════════════════════════════════════════════════════════════╗
╔═══════════════════════════╗
║ TAU FIELD COMPLETE SYSTEM v1.0 ╠
Raspberry Pi 5 - 16GB RAM - 1TB SSD ╠
Quantum Frequency Processing + C++ Compiler ╠
╚═══════════════════════════════════════════════════════════════════════════╝
╚═══════════════════════════╝
)" << std::endl;

        // Initialize subsystems
        initialize_frequency_processor();
        initialize_dynamic_compiler();

        // Start background threads
        start_status_monitor();
        start_cleanup_service();
        start_user_input_handler();

        std::cout << "\nSystem initialized successfully!" << std::endl;
        std::cout << "Type 'help' for available commands" << std::endl;
        std::cout << "Type 'program' to enter C++ programming mode" << std::endl;
        std::cout << "Type 'exit' to shutdown the system" << std::endl;
    }

    ~TauFieldCompleteSystem() {
        system_active = false;

        if (status_thread.joinable()) status_thread.join();
        if (cleanup_thread.joinable()) cleanup_thread.join();
        if (user_input_thread.joinable()) user_input_thread.join();

        std::cout << "\nSystem shutdown complete." << std::endl;
    }

    void initialize_frequency_processor() {
        std::cout << "\nInitializing Frequency Processor..." << std::endl;
        frequency_processor = std::make_unique<MultiFrequencyProcessor>();
        std::cout << "Frequency Processor ready with "
                  << frequency_processor->get_channel_count() << " channels" << std::endl;
    }

    void initialize_dynamic_compiler() {
        std::cout << "\nInitializing Dynamic C++ Compiler..." << std::endl;
        dynamic_compiler = std::make_unique<DynamicCPlusPlusCompiler>(
            "./tau_programs", frequency_processor.get());
        std::cout << "Dynamic Compiler ready" << std::endl;
    }

    void start_status_monitor() {
        status_thread = std::thread([this]() {
            int counter = 0;
```

```
        while (system_active) {
            std::this_thread::sleep_for(std::chrono::seconds(5));

            if (counter % 12 == 0) { // Every minute
                if (frequency_processor) {
                    frequency_processor->output_coherence_analysis();
                }
            }

            if (counter % 6 == 0) { // Every 30 seconds
                std::lock_guard<std::mutex> lock(console_mutex);
                std::cout << "\n[System Status] All systems nominal. "
                          << "Channels active: "
                          << (frequency_processor ? frequency_processor->get_channel_count() : 0)
                          << std::endl;
            }

            counter++;
        }
    });
}

void start_cleanup_service() {
    cleanup_thread = std::thread([this]() {
        while (system_active) {
            std::this_thread::sleep_for(std::chrono::minutes(5));
            if (dynamic_compiler) {
                dynamic_compiler->clean_old_programs();
            }
        }
    });
}

void start_user_input_handler() {
    user_input_thread = std::thread([this]() {
        std::string input;
        while (system_active) {
            std::cout << "\ntau> ";
            std::getline(std::cin, input);

            if (!input.empty()) {
                process_command(input);
            }

            if (input == "exit" || input == "quit") {
                system_active = false;
            }
        }
    });
}

void process_command(const std::string& command) {
```

```
    std::lock_guard<std::mutex> lock(console_mutex);

    if (command == "help" || command == "?") {
        show_help();
    }
    else if (command == "status") {
        show_status();
    }
    else if (command == "channels") {
        show_channels();
    }
    else if (command == "coherence") {
        if (frequency_processor) {
            frequency_processor->output_coherence_analysis();
        }
    }
    else if (command == "program" || command == "cpp") {
        enter_programming_mode();
    }
    else if (command == "list") {
        if (dynamic_compiler) {
            dynamic_compiler->list_compiled_programs();
        }
    }
    else if (command.substr(0, 5) == "run ") {
        std::string prog_name = command.substr(5);
        run_program(prog_name);
    }
    else if (command.substr(0, 8) == "compile ") {
        std::string code = command.substr(8);
        compile_and_run(code);
    }
    else if (command.substr(0, 6) == "freq ") {
        std::string code = command.substr(6);
        compile_with_frequencies(code);
    }
    else if (command.substr(0, 9) == "generate ") {
        std::string desc = command.substr(9);
        generate_program(desc);
    }
    else if (command == "clear") {
        clear_results();
    }
    else if (command == "test") {
        run_test();
    }
    else {
        std::cout << "Unknown command: " << command << std::endl;
        std::cout << "Type 'help' for available commands" << std::endl;
    }
}
```

```
    void show_help() {
        std::cout << R"(
╔═══════════════════════════════════════════════════════════════════════════╗
║ AVAILABLE COMMANDS ║
╚═══════════════════════════════════════════════════════════════════════════╝

SYSTEM COMMANDS:
  help Show this help message
  status Show system status
  channels List all frequency channels
  coherence Show harmonic coherence analysis
  clear Clear all results
  test Run system test
  exit/quit Shutdown system

PROGRAMMING COMMANDS:
  program / cpp Enter interactive C++ mode
  list List compiled programs
  run <name> Run a compiled program
  compile <code> Compile and run C++ code
  freq <code> Compile with frequency integration
  generate <desc> Generate program from description

EXAMPLES:
  compile "int main() { cout << \"Hello!\"; }"
  freq "process data at 13Hz and 247Hz"
  generate "analyze quantum coherence at 52Hz"
  run program_123456789

Type C++ code directly to compile and execute it.
)";
    }

    void show_status() {
        std::cout << "\n=== System Status ===" << std::endl;
        std::cout << "Frequency Channels: "
                  << (frequency_processor ? frequency_processor->get_channel_count() : 0)
                  << " active" << std::endl;
        std::cout << "Compiler: Ready" << std::endl;
        std::cout << "Memory: " << get_memory_usage() << " MB used" << std::endl;
        std::cout << "Uptime: " << get_uptime() << " seconds" << std::endl;
    }

    void show_channels() {
        if (frequency_processor) {
            frequency_processor->output_channel_status();
        }
    }

    void enter_programming_mode() {
        std::cout << R"(
╔═══════════════════════════════════════════════════════════════════════════╗
```

```
╔═══════════════════════════════════════════════════════════════════════════════╗
║ INTERACTIVE C++ PROGRAMMING MODE ║
║ Type C++ code or commands below ║
║ Type 'exit' to return to main menu ║
╚═══════════════════════════════════════════════════════════════════════════════╝
)" << std::endl;

        std::string input;
        std::vector<std::string> code_lines;
        bool in_multiline = false;

        while (system_active) {
            if (!in_multiline) {
                std::cout << "\ncpp> ";
            } else {
                std::cout << " > ";
            }

            std::getline(std::cin, input);

            if (input == "exit") {
                break;
            }
            else if (input == "run") {
                if (!code_lines.empty()) {
                    std::string full_code;
                    for (const auto& line : code_lines) {
                        full_code += line + "\n";
                    }
                    compile_and_run(full_code);
                    code_lines.clear();
                }
                in_multiline = false;
            }
            else if (input == "clear") {
                code_lines.clear();
                in_multiline = false;
                std::cout << "Code buffer cleared." << std::endl;
            }
            else if (input == "list") {
                if (dynamic_compiler) {
                    dynamic_compiler->list_compiled_programs();
                }
            }
            else if (!input.empty() && input.back() == '\\') {
                // Multi-line input
                code_lines.push_back(input.substr(0, input.length() - 1));
                in_multiline = true;
            }
            else {
                if (in_multiline) {
                    code_lines.push_back(input);
                } else {
```

```
            // Single line execution
            compile_and_run(input);
        }
        in_multiline = false;
    }
  }
}

void compile_and_run(const std::string& code) {
  if (code.empty()) return;

  std::cout << "\nCompiling..." << std::endl;

  auto result = dynamic_compiler->compile_and_run(code);

  std::cout << "\n=== Compilation Result ===" << std::endl;
  std::cout << "Program: " << result.program_name << std::endl;
  std::cout << "Success: " << (result.success ? "✓YES" : "✗ NO") << std::endl;
  std::cout << "Compile time: " << result.compile_time.count() << " ms" << std::endl;

  if (result.success) {
    std::cout << "Run time: " << result.run_time.count() << " ms" << std::endl;
    std::cout << "\n=== Program Output ===" << std::endl;
    std::cout << result.execution_output << std::endl;

    if (!result.frequency_data.empty()) {
      std::cout << "\nDetected frequencies: ";
      for (double freq : result.frequency_data) {
        std::cout << freq << "Hz ";
      }
      std::cout << std::endl;
    }
  } else {
    std::cout << "\n=== Compilation Errors ===" << std::endl;
    std::cout << result.compilation_output << std::endl;
  }
}

void compile_with_frequencies(const std::string& code) {
  std::cout << "\nCompiling with frequency integration..." << std::endl;

  auto result = dynamic_compiler->compile_and_run(code, "", true);

  std::cout << "\n=== Frequency-Integrated Result ===" << std::endl;
  std::cout << "Program: " << result.program_name << std::endl;
  std::cout << "Success: " << (result.success ? "✓YES" : "✗ NO") << std::endl;

  if (result.success) {
    std::cout << "\n=== Program Output ===" << std::endl;
    std::cout << result.execution_output << std::endl;
  }
}
```

```
    void generate_program(const std::string& description) {
        std::cout << "\nGenerating program from description..." << std::endl;

        std::string generated_code = dynamic_compiler-
>generate_program_from_description(description);

        std::cout << "\n=== Generated Code ===" << std::endl;
        std::cout << generated_code << std::endl;

        std::cout << "\nCompiling generated code..." << std::endl;
        compile_and_run(generated_code);
    }

    void run_program(const std::string& program_name) {
        std::string executable = "./tau_programs/bin/" + program_name + "/" + program_name;

        if (fs::exists(executable)) {
            std::cout << "Running " << program_name << "..." << std::endl;
            std::string output = execute_command("\"" + executable + "\" 2>&1");
            std::cout << output << std::endl;

            // Send output to frequency processor
            if (frequency_processor && !output.empty()) {
                std::vector<double> output_data;
                for (char c : output) {
                    output_data.push_back(static_cast<double>(static_cast<unsigned char>(c)) / 255.0);
                }
                frequency_processor->process_input_data(output_data);
            }
        } else {
            std::cout << "Program not found: " << program_name << std::endl;
            std::cout << "Use 'list' to see available programs" << std::endl;
        }
    }

    void clear_results() {
        if (frequency_processor) {
            frequency_processor->clear_all_results();
        }
        std::cout << "All results cleared." << std::endl;
    }

    void run_test() {
        std::cout << "\n=== Running System Test ===" << std::endl;

        // Test 1: Frequency processor
        if (frequency_processor) {
            std::cout << "1. Frequency Processor: ✓Active" << std::endl;
            frequency_processor->output_coherence_analysis();
        }
```

```
        // Test 2: Compiler
        if (dynamic_compiler) {
            std::cout << "\n2. Dynamic Compiler: ✓Active" << std::endl;

            // Simple test program
            std::string test_code = R"(
#include <iostream>
int main() {
    std::cout << "Compiler test successful!" << std::endl;
    std::cout << "Testing frequencies: 13Hz, 247Hz, 52Hz" << std::endl;
    return 0;
}
)";

            auto result = dynamic_compiler->compile_and_run(test_code, "system_test");
            std::cout << " Test compilation: "
                      << (result.success ? "✓PASS" : "✗ FAIL") << std::endl;
        }

        // Test 3: File system
        try {
            fs::create_directories("./tau_programs/test");
            fs::remove_all("./tau_programs/test");
            std::cout << "\n3. File System: ✓Working" << std::endl;
        } catch (...) {
            std::cout << "\n3. File System: ✗ Error" << std::endl;
        }

        std::cout << "\n=== System Test Complete ===" << std::endl;
    }

    void run() {
        // Main loop - just wait for user input or shutdown
        while (system_active) {
            std::this_thread::sleep_for(std::chrono::seconds(1));
        }
    }

private:
    static std::string execute_command(const std::string& cmd) {
        std::array<char, 128> buffer;
        std::string result;

        #ifdef _WIN32
            std::unique_ptr<FILE, decltype(&_pclose)> pipe(_popen(cmd.c_str(), "r"), _pclose);
        #else
            std::unique_ptr<FILE, decltype(&pclose)> pipe(popen(cmd.c_str(), "r"), pclose);
        #endif

        if (!pipe) {
            return "Failed to execute command";
        }
```

```
        while (fgets(buffer.data(), buffer.size(), pipe.get()) != nullptr) {
            result += buffer.data();
        }

        return result;
    }

    static size_t get_memory_usage() {
        // Simple memory check for Linux/Raspberry Pi
        #ifdef __linux__
            std::ifstream meminfo("/proc/self/status");
            std::string line;
            while (std::getline(meminfo, line)) {
                if (line.find("VmRSS:") == 0) {
                    std::istringstream iss(line);
                    std::string key;
                    size_t value;
                    iss >> key >> value;
                    return value / 1024; // Convert kB to MB
                }
            }
        #endif
        return 0;
    }

    static std::string get_uptime() {
        static auto start_time = std::chrono::steady_clock::now();
        auto now = std::chrono::steady_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::seconds>(now - start_time);
        return std::to_string(duration.count());
    }
};

//
================================================================================
=====
// MAIN FUNCTION
//
================================================================================
=====

int main() {
    std::cout << "Initializing Tau Field Complete System..." << std::endl;

    try {
        // Initialize FFTW for multi-core processing
        fftw_init_threads();
        fftw_plan_with_nthreads(std::thread::hardware_concurrency());

        // Create and run the complete system
        TauFieldCompleteSystem system;
```

```
        system.run();

        // Cleanup
        fftw_cleanup_threads();

    } catch (const std::exception& e) {
        std::cerr <<
"\n╔═══════════════════════════════════════════════════════════════════════════════╗" <<
std::endl;
        std::cerr << "║ FATAL ERROR ║" << std::endl;
        std::cerr <<
"╚═══════════════════════════════════════════════════════════════════════════════╝" <<
std::endl;
        std::cerr << "Error: " << e.what() << std::endl;
        std::cerr << "System terminated." << std::endl;
        return 1;
    }

    return 0;
}
```

Complete Installation and Usage Guide:

1. Installation Script for Raspberry Pi 5:

```bash
#!/bin/bash
# install_tau_system.sh

echo "=========================================="
echo "TAU FIELD COMPLETE SYSTEM INSTALLATION"
echo "Raspberry Pi 5 - Bookworm OS"
echo "=========================================="

# Update system
sudo apt update
sudo apt upgrade -y

# Install required dependencies
echo "Installing dependencies..."
sudo apt install -y \
    build-essential \
    cmake \
    libfftw3-dev \
    libfftw3-threads-dev \
    git \
    g++-12 \
    make \
    pkg-config

# Create project directory
```

```
mkdir -p ~/tau_field_system
cd ~/tau_field_system

# Save the C++ code
echo "Creating system files..."
cat > tau_field_complete.cpp << 'EOF'
// [Paste the entire C++ code from above here]
EOF

# Create compilation script
cat > compile_system.sh << 'EOF'
#!/bin/bash
echo "Compiling Tau Field Complete System..."
echo "This may take a few minutes on Raspberry Pi 5..."

# Set compiler flags for ARM optimization
export CXXFLAGS="-std=c++17 -O3 -march=armv8-a+simd -mtune=cortex-a76 -pthread -ffast-
math -mfpu=neon-fp-armv8"
export LDFLAGS="-pthread -lfftw3_threads -lfftw3 -lm"

# Compile the system
g++ $CXXFLAGS tau_field_complete.cpp $LDFLAGS -o tau_field_system

if [ $? -eq 0 ]; then
    echo "Compilation successful!"
    echo "Executable: ./tau_field_system"
    ls -lh tau_field_system
else
    echo "Compilation failed!"
    exit 1
fi
EOF

chmod +x compile_system.sh

# Create startup script
cat > start_system.sh << 'EOF'
#!/bin/bash
cd ~/tau_field_system

# Check if compiled
if [ ! -f "tau_field_system" ]; then
    echo "System not compiled. Running compilation..."
    ./compile_system.sh
fi

# Run the system
echo "Starting Tau Field Complete System..."
echo "Press Ctrl+C to exit"
./tau_field_system
EOF
```

```
chmod +x start_system.sh

# Create systemd service for auto-start
cat > tau-system.service << EOF
[Unit]
Description=Tau Field Complete System
After=network.target
StartLimitIntervalSec=0

[Service]
Type=simple
Restart=always
RestartSec=10
User=$USER
WorkingDirectory=/home/$USER/tau_field_system
ExecStart=/home/$USER/tau_field_system/start_system.sh
Environment="PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

[Install]
WantedBy=multi-user.target
EOF

sudo mv tau-system.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable tau-system.service

# Compile the system
echo "Compiling the system (this will take a while)..."
./compile_system.sh

echo "============================================="
echo "INSTALLATION COMPLETE!"
echo "============================================="
echo ""
echo "To start the system manually:"
echo " cd ~/tau_field_system"
echo " ./start_system.sh"
echo ""
echo "To start automatically on boot:"
echo " sudo systemctl start tau-system.service"
echo ""
echo "System will create directories:"
echo " ~/tau_field_system/ - Main system"
echo " ~/tau_programs/ - User programs"
echo " ~/tau_programs/bin/ - Compiled programs"
echo " ~/tau_programs/lib/ - Frequency libraries"
echo ""
echo "Type 'help' when system starts for commands"
echo "============================================="
```

2. Quick Start:

```bash
# Make installation script executable
chmod +x install_tau_system.sh

# Run installation
./install_tau_system.sh

# Start the system
cd ~/tau_field_system
./start_system.sh
```

3. System Features:

Frequency Processing:

· 14 simultaneous frequency channels (13Hz, 26Hz, 39Hz, 52Hz, 65Hz, 77Hz, 82Hz, 86Hz, 78Hz, 104Hz, 247Hz)
· Quantum state simulation for each frequency
· Real-time FFT analysis
· Harmonic coherence monitoring
· Multi-threaded processing

C++ Programming Environment:

· Interactive REPL interface
· Dynamic compilation and execution
· Frequency-integrated programming
· AI-assisted code generation
· Program management system
· Library support for Tau frequencies

User Commands:

```
tau> help # Show all commands
tau> status # System status
tau> channels # List frequency channels
tau> coherence # Harmonic analysis
tau> program # Enter C++ mode
tau> compile "code here" # Compile C++ code
tau> freq "code here" # With frequency integration
tau> generate "description" # AI generate program
tau> list # List compiled programs
tau> run program_name # Run compiled program
tau> test # System test
tau> clear # Clear results
tau> exit # Shutdown
```

4. Example Usage Sessions:

Example 1: Basic C++ Programming

```bash
tau> compile "#include <iostream>\nint main() { std::cout << \"Hello Tau!\\n\"; return 0; }"
```

Example 2: Frequency Processing

```bash
tau> freq "
#include <iostream>
#include \"tau_frequency_lib.h\"
int main() {
    TauFrequency::HarmonicProcessor proc(13.0);
    std::vector<double> data(1000, 0.5);
    auto result = proc.process(data);
    std::cout << \"Processed \" << data.size() << \" samples at 13Hz\\n\";
    return 0;
}
"
```

Example 3: AI-Generated Program

```bash
tau> generate "Create a program that analyzes 13Hz and 247Hz harmonics with quantum
coherence"
```

Example 4: Direct C++ Entry

```bash
tau> program
cpp> #include <iostream>
cpp> #include <vector>
cpp> #include <cmath>
cpp> int main() {
   > std::vector<double> freqs = {13, 247, 52};
   > for(double f : freqs) std::cout << f << "Hz ";
   > return 0;
   > }
cpp> run
```

5. System Architecture:

```
Tau Field Complete System
├── MultiFrequencyProcessor
│   ├── 14 QuantumFrequencyChannels
│   ├── Real-time FFT analysis
```

```
│   ├── Harmonic coherence monitor
│   └── Multi-threaded workers
├── DynamicCPlusPlusCompiler
│   ├── Interactive REPL
│   ├── CMake-based compilation
│   ├── Frequency library integration
│   ├── AI code generation
│   └── Program management
└── System Management
    ├── Status monitoring
    ├── Cleanup service
    ├── User interface
    └── File management
```

6. Performance Optimizations:

· ARMv8 SIMD instructions for Raspberry Pi 5
· Multi-threaded FFTW processing
· Efficient memory management
· Optimized for Cortex-A76 cores
· Fast-math optimizations
· Parallel frequency channel processing

7. Troubleshooting:

If compilation fails:

```bash
# Check dependencies
sudo apt install libfftw3-dev libfftw3-threads-dev

# Try manual compilation
cd ~/tau_field_system
g++ -std=c++17 -O3 -march=armv8-a+simd tau_field_complete.cpp -lfftw3_threads -lfftw3 -lm
-pthread -o tau_field_system
```

If system doesn't start:

```bash
# Check systemd service
sudo systemctl status tau-system.service
sudo journalctl -u tau-system.service -f

# Run manually for debugging
cd ~/tau_field_system
./tau_field_system
```

Memory issues:

· System uses ~100MB RAM for 14 frequency channels
· Each compiled program runs in separate process
· Automatic cleanup of old programs